# THE NATIONAL UNIVERSITY OF LESOTHO

**Faculty of Science and Technology — Department of Mathematics and Computer Science**

**CS3520 — Computer Organisation and Architecture I**

**PRACTICAL LAB HANDOUT**

| | |
|---|---|
| **Lab No.** | Lab 2 |
| **Lab Title** | Assembly Programming II — Control Flow, Procedures & Your Own Repository |
| **Supports Week(s)** | Weeks 3–4 — instruction formats and encoding; branches, loops, procedures, the stack and calling conventions |
| **Duration** | 2 hours scheduled, plus completion in your own time before the deadline |
| **Venue** | ICT Lab |
| **Submission Platform** | Thuto — submit the URL of your public GitHub repository. The work itself lives in the repository, not in an upload. |
| **Weighting** | Continuous lab assessment |

## LEARNING OUTCOMES

On completion of this lab, you should be able to:

– Maintain a public course repository of your own, with a clear and readable commit history. (CLO7) – Model each problem in C++ and test it before committing to an assembly implementation. (CLO2) – Implement solutions using conditional branches, counted loops, procedures and the stack, respecting the calling convention. (CLO2)

– Identify and decode B-type and J-type instructions in the machine code your programs generate. (CLO2)

– Document your work so that another engineer can follow your reasoning without asking you. (CLO8)

## BEFORE YOU START

– Lab 1 completed — you can already translate straight-line C++ into assembly and decode an instruction by hand.

– A GitHub account. This lab requires one: your work is assessed from your own public repository. – The Week 3 and Week 4 lectures. You will need branches, loops, jal/jalr, and the register conventions throughout.

## TOOLS & RESOURCES

| Tool / Resource | Purpose |
|---|---|
| Online C++ compiler | Testing each model quickly |
| Ripes | Assembling, running and inspecting the generated machine code |
| Git and GitHub | Your own public repository — where this lab is submitted from |
| Course repository | github.com/khobatha/CS3520 — the worked example studied in Step 2 |

| | |
|---|---|
| RISC-V reference card | Opcode and funct values, including the branch and jump formats |

## PROCEDURE

### Step 1 — Create Your Own Public Course Repository

In Labs 0 and 1 you cloned the course repository directly, which is read-only for you. From this lab onwards you work in your own copy, so that your solutions can be committed and assessed.

– On GitHub, fork github.com/khobatha/CS3520 into your own account. Keep it public. – Clone your fork, and add the course repository as an upstream remote so you can still receive new material:

```
git clone https://github.com/<your-username>/CS3520.git
cd CS3520
git remote add upstream https://github.com/khobatha/CS3520.git
git fetch upstream
git merge upstream/main
```

> Run the last two commands at the start of every future lab. Your fork does not update itself — fetching from upstream is how new examples reach you.

### Step 2 — Study the Worked Example

Open examples/2026/array-max/. It contains the same algorithm expressed twice: a tested C++ model, and its RISC-V translation.

– Run model.cpp in an online C++ compiler. Note the output.

– Assemble and run array-max.s in Ripes. Confirm it prints the same thing.

– Step through the assembly and watch s1, t1 and sp change as the loop runs.

Then answer these in your notes — they are the point of the exercise, not the running of it: – The C++ loop says i < n, but the assembly branches on bge (i >= n). Why must the test be inverted? – li, mv, la and ble are pseudo-instructions. Compare your source with the machine code Ripes generates — what real instructions does each become, and why does the assembler provide them at all? – find_max saves s1 on the stack but does not save ra. Explain why both decisions are correct.

### Step 3 — Prepare Your Lab 2 Workspace

In your fork, create the following structure. One folder per exercise, each self-contained:

```
labs/2026/lab-2/
├── ex1-larger-of-two/
│   ├── model.cpp
│   ├── solution.s
│   └── notes.md
├── ex2-sum-to-n/
└── ... (one folder per exercise below)
```

Each notes.md should be short — half a page at most — and state: what the program does, which register holds what, and anything that was harder than expected.

### Step 4 — Solve the Problem Set

Work through these in order. Each becomes its own folder, and each follows the same discipline: write the C++ model first, test it in an online compiler, then translate it by hand and verify the assembly produces an identical result.

| Ex. | Problem | Constructs you should use |
|---|---|---|

| | | |
|---|---|---|
| **1** | Print the larger of two integers held in the .data section. | Conditional branch |
| **2** | Compute the sum of the first N integers, for N stored in memory. | Counted loop, branch |

| | | |
|---|---|---|
| **3** | Count how many elements of an array are even, and print the count. | Loop, branch, andi for the bit test, array indexing |
| **4** | Compute the factorial of N, with the calculation written as a procedure that main calls. | jal / jalr, argument and return registers, the stack |
| **5** | Challenge: compute the greatest common divisor of two integers using Euclid's algorithm, written as a procedure. | Procedure with a loop, repeated subtraction or remainder, careful register discipline |

Stretch, if you finish early: rewrite Exercise 5 so the procedure calls itself recursively instead of looping. It will then no longer be a leaf procedure — work out what that changes about the stack, and record the answer in your notes.

### Step 5 — Analyse the New Instruction Formats

In Lab 1 you decoded R-type, I-type and S-type instructions. Your solutions now contain two formats you have not yet taken apart. Choose one branch and one jump from any of your exercises and, for each:

– Record the 32-bit encoding Ripes generates.

– Split it into its fields and identify the format — B-type for the branch, J-type for the jump. – Explain, in a sentence, why the immediate is stored in scattered pieces rather than as one contiguous field.

Put this analysis in labs/2026/lab-2/encoding-analysis.md.

### Step 6 — Commit Your Work Properly

Commit as you go, not all at once at the end. Aim for at least two commits per exercise — one for the tested model, one for the working assembly — with messages that say what changed and why:

```
git add labs/2026/lab-2/ex2-sum-to-n/model.cpp
git commit -m "ex2: add C++ model for sum of first N integers"

git add labs/2026/lab-2/ex2-sum-to-n/solution.s
git commit -m "ex2: add RISC-V implementation; loop exits on bge as in worked example"

git push origin main
```

A good message names the exercise and describes the change. "update", "fix", "final" and "asdf" tell a reader nothing and will cost marks.

## WHAT TO SUBMIT

Submit the URL of your public GitHub repository on Thuto. Before you do, check that it contains: – labs/2026/lab-2/ with one folder per exercise, each holding model.cpp, solution.s and notes.md – Your answers to the Step 2 discussion questions — put them in labs/2026/lab-2/README.md – encoding-analysis.md from Step 5

– A commit history that shows the work progressing, with meaningful messages

Assessment looks at the repository as a whole: working code, honest notes, and a history that someone else can follow. A single commit containing everything, added minutes before the deadline, will not score well even if the code runs.

## GETTING HELP

– Lab demonstrators are on hand throughout the session — ask before searching online.

– Outside lab hours: post on the CS3520 course forum, or email ko.setetemela@nul.ls. – Consultation hours are posted on the office notice board.